A Project Report on

AgriAgent: A Safety-Gated Zero-Shot Vision Pipeline for Crop–Weed Spot Spraying

Submitted in partial fulfilment of the requirements
for the degree of

Bachelor of Technology in Computer Engineering

by

Chaitanya Shinde	**231070066**
Sumant Vetal	*roll no.*
Sahil Chavan	*roll no.*
Amit Ingle	*roll no.*

Under the guidance of

Prof. V. D. Dhore

Department of Computer Engineering and Information Technology
Veermata Jijabai Technological Institute (VJTI)
Mumbai 400 019

Academic year 2026–27

Certificate

This is to certify that the project entitled **"AgriAgent: A Safety-Gated Zero-Shot Vision Pipeline for Crop–Weed Spot Spraying"** is a bonafide record of the work carried out by Chaitanya Shinde (231070066), Sumant Vetal, Sahil Chavan and Amit Ingle, submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Engineering of Veermata Jijabai Technological Institute, Mumbai, during the academic year 2026–27.

Prof. V. D. Dhore Project Guide	Head of Department Computer Engineering and IT
Internal Examiner	External Examiner

Date: Place: Mumbai

Declaration

We declare that this report expresses our ideas in our own words, and where the ideas or words of others are included, we have cited and referenced the original sources. Every number reported here was produced by the code in the project repository on the hardware described in Chapter 4, and the file holding each result is named next to it. We declare that we have not misrepresented, fabricated or falsified any idea, data, fact or source. Where earlier drafts of this project contained claims we could not verify, they were withdrawn, and Section 7.1 records what was withdrawn and why.

Chaitanya Shinde (231070066)

Sumant Vetal

Sahil Chavan

Amit Ingle

Acknowledgement

We thank our guide, Prof. V. D. Dhore, for agreeing to a narrower, measurable scope when our first real results showed that the original proposal could not be proven in one semester, and for asking the question that shaped the final report: did we implement the base paper, and did the escalation work? We thank the Department of Computer Engineering and IT, VJTI, for the facilities. We thank the authors who released their models, code and data openly: IDEA-Research (Grounding DINO), Meta AI (SAM 2), the University of Bonn Photogrammetry and Robotics Lab (sugar beet data and the WeedsAreWeird code), Mississippi State University (CottonWeedDet3) and ICAR–Directorate of Weed Research (the herbicide bulletin).

Abstract

Spot spraying puts herbicide only on weeds instead of on the whole field, but it needs a vision system that can tell a weed from a crop plant. Supervised networks do this well after training on labelled images of the target field; zero-shot foundation models promise to do it without any training. **AgriAgent** tests that promise honestly. It implements the Grounded SAM pipeline [1] (a text-prompted Grounding DINO detector [2] followed by the SAM 2 segmenter [3]), adds a deterministic safety gate as the final authority on what may be sprayed, a rule-based plant classifier, one bounded escalation agent that consults vision-language models about uncertain plants, and herbicide advice grounded in the ICAR herbicide bulletin. It runs on a laptop GPU (RTX 5050, 8 GB) in about 0.6 s per photo.

We evaluate on sugar beet and cotton photos with labels copied from the official dataset files, after discovering and replacing an earlier evaluation set whose labels had been fabricated. The measured answer to our first research question is negative for text prompts alone: on 90 held-out sugar beet photos, the Grounded SAM label policy sprays **65.5% of crop pixels** while hitting only **1.8% of weed pixels**. A frozen set of four plain rules (crop-mask overlap, planting row, size relative to the crop) raises weed recall to **77.9%** while cutting crop pixels sprayed to **8.8%**, measured once on the held-out set. The escalation agent answers our second question: across 69 reviewed plants it never approved spraying a crop plant and its 14 "crop" verdicts were 13 times right, but it called no weed a weed, so it is safe but not effective. The best published zero-shot method reaches 12.1% weed IoU against 62.7% for a supervised network [4]; our results agree that zero-shot prompting does not separate crop from weed, and show how much a transparent safety layer recovers. The system is a research prototype and must not drive a real sprayer.

Keywords: precision agriculture, weed detection, zero-shot segmentation, Grounding DINO, SAM 2, safety gate, vision-language models, spot spraying.

Contents

1 Introduction 1
1.1 Motivation 1
1.2 Problem statement 1
1.3 Research questions 1
1.4 Objectives 1
1.5 Scope 2
1.6 Contributions 2
1.7 Organisation of the report 2
2 Literature review 3
2.1 Supervised crop–weed segmentation 3
2.2 Open-vocabulary detection and promptable segmentation 3
2.3 Zero-shot crop–weed segmentation 3
2.4 Vision-language models in agriculture 3
2.5 Base paper and research gap 4
3 Requirements and system design 5
3.1 Requirements 5
3.2 Architecture 5
3.3 The safety gate 6
3.4 Plant rules (v3, frozen) 7
3.5 The escalation agent 7
3.6 System 2 and ICAR herbicide advice 8
3.7 Farmer summary 8
4 Implementation 9
4.1 Technology stack and hardware 9
4.2 Code structure 9
4.3 Real and mock modes 10
4.4 Application 10
4.5 GeoJSON export 10
5 Data and evaluation method 11
5.1 Datasets 11
5.2 A fabricated evaluation set, found and replaced 11
5.3 Development and held-out sets 12
5.4 Metrics 12
5.5 Settings compared 12
6 Results 13
6.1 Baseline: Grounded SAM label policy (RQ1) 13
6.2 Cheap additions: crop exclusion, lower thresholds, tiling (RQ3) 13
6.3 Plant rules on the development set 14
6.4 Held-out result (the headline) 14
6.5 Escalation agent (RQ2) 15
6.6 Vision-language models as a second opinion 15
6.7 Main limitation: rule 4 and whole beets (F1) 15
6.8 ICAR herbicide table check 16
6.9 Speed 16

7 Testing 17
7.1 Test levels 17
7.2 System test cycle 0 17
7.3 Findings and fixes 18
7.4 Manual test cycle 20
7.5 System test cycle 2 and the release 20
8 Discussion and conclusion 21
8.1 What we learned, including what went wrong 21
8.2 Answers to the research questions 21
8.3 Relation to the base paper and the closest work 21
8.4 Limitations 21
8.5 Future work 22
8.6 Conclusion 22
References 23
Appendix A: How to run 25
Appendix B: Team contributions 26

List of Figures

Figure 1 AgriAgent pipeline. Green: local and deterministic or pretrained; yellow: the safety gate; blue: cloud models that advise only. 6
Figure 2 Four-panel view of a real run on a sugar beet photo (17 October, after the gate fix): photo, detector boxes, masks, and the spray map. All detector confidences were 0.36–0.47, below the 0.70 gate, so nothing was sprayed: the crop is safe, but the weeds are missed. This is the Grounded SAM baseline behaviour that the plant rules address. . 10
Figure 3 Label audit, beet X-10-0: the old labels (left) put a "crop" box on bare soil and "weed" boxes on beets; official labels (right). 11
Figure 4 Label audit, cotton CM_17: no old box matches the official weed boxes; the official dataset has no cotton labels. 11
Figure 5 Crop pixels sprayed against weed recall on the held-out set. Every Grounded SAM setting lies on the left edge (almost no weeds hit); the plant rules move the operating point to 77.9% recall at 8.8% crop sprayed. 14
Figure 6 Finding F1. Photo X-10-0 rotated by 90°: the detector returns no crop box, so rule 4 marks the largest beet as an off-row weed and it is sprayed completely. 16

List of Tables

Table 1 Research questions. 1
Table 2 Related work and where AgriAgent sits. 4
Table 3 Functional (FR) and non-functional (NFR) requirements. 5
Table 4 Technology stack. 9
Table 5 Main modules. 9
Table 6 Data sources. 11
Table 7 Evaluation sets. All labels are copied unchanged from the official files. 12
Table 8 Settings compared in Chapter 6. Prompt `weed . <crop> plant .`, detector box threshold 0.35, buffer 10 px. 12
Table 9 Sugar beet development set (30 photos). Source: `docs/results/baseline_v2_20261018.md`. 13
Table 10 Crop exclusion and detector variants, sugar beet development set. Sources: `exclusion_v2_20261018.md`, `detector_variants_20261018.md`. 13
Table 11 Plant rules v3, sugar beet development set. Sources: `plant_rules_v3_20261018.md`, `pipeline_plant_rules_dev_20261020.md` (the product's own code path gives identical numbers). 14
Table 12 Held-out sugar beet set (90 photos, evaluated once). Source: `holdout_plant_rules_v3_20261018.md`. 14
Table 13 Verdicts of the escalation agent on 69 withheld plants, development set, deployed gate. Source: `agent_v2_20261018.md`. 15
Table 14 Crop-or-weed answers on 20 labelled plant patches. Sources: `t2_2_crop_or_weed_groq_20261017.json`, `qwen_provider_check_20261030.md`. 15
Table 15 The size limit trades recall for crop safety. Development set. Source: `scripts/experiment_f1_row_rule.py`, Verification report 23 Oct. 16
Table 16 Testing summary. 17

Table 17 Selected findings (full list: `docs/testing/Verification_Report_20261023.md`, `Manual_Test_Cycle_20261030.md`). 19
Table 18 Contributions counted from the repository. 26
Table 19 Components and their owners (owner = who built the current version). 26

Chapter 1

Introduction

1.1 Motivation

Herbicide is usually applied uniformly across a field, although weeds cover only part of it. Spot spraying, where a camera finds the weeds and a nozzle opens only over them, can cut herbicide use substantially, but only if the camera system separates weeds from the crop: a false "weed" on a crop plant damages the crop, and a missed weed survives. In practice this separation is done by supervised networks trained on thousands of labelled images of the same crop and field conditions. Collecting those labels is expensive, and a network trained on one field often fails on another.

Foundation models offer a different route. Grounding DINO [2] finds objects named in a free-text prompt, and SAM 2 [3] outlines any object it is pointed at. Combined, as in Grounded SAM [1], they segment "the weeds" in a photo without a single training label. Whether that actually works for crop and weed, which are both green plants, often of similar shape, is an open question. A recent study by Chong et al. [4] argues that text prompts are ill-defined for this task. AgriAgent measures it on real field photos, and asks what a transparent safety layer can recover.

1.2 Problem statement

Given an RGB field photo and the name of the crop, produce a spray map that covers the weeds and leaves the crop untouched, using pretrained models only (no training on our data), on a laptop GPU, with every spraying decision explainable and testable.

1.3 Research questions

Table 1: Research questions.

ID	Question
RQ1	How well do zero-shot detection and segmentation (Grounded SAM) separate weed from crop? Measured as weed recall and the share of crop pixels that end up on the spray map.
RQ2	When the deterministic gate is unsure and asks vision-language models (escalation), does the spray map become more complete without becoming less safe?
RQ3	Which cheap addition reduces crop on the map most (a stricter confidence gate, subtracting crop masks, rules on planting rows and plant size), and what does it cost in missed weeds?

1.4 Objectives

1. Run Grounding DINO and SAM 2 on our own laptop GPU, zero-shot, on sugar beet and cotton photos.
2. Build a deterministic safety gate that is the only authority on what is sprayed, and that withholds anything uncertain.
3. Measure crop pixels sprayed and weed recall on photos with official labels, with a held-out set used once.
4. Test one bounded escalation agent on the plants the gate withholds.
5. Ground any herbicide advice in the ICAR bulletin for the declared crop, never invent a dose.

6. Deliver a demonstrable application (Streamlit) with a GeoJSON spray map export and a plain-language summary for a farmer in English, Hindi or Marathi.

1.5 Scope

AgriAgent is a research prototype. It produces a map on screen and a file; it does not drive a sprayer, a drone or a tractor, and its GeoJSON coordinates are placeholders, not GPS. Weed versus crop for spot spraying is the only application. Disease, yield counting, canopy cover and nutrient stress, listed in our original proposal, are future work (Section 8.2). No model is trained or fine-tuned.

1.6 Contributions

1. An honest measurement of the Grounded SAM pipeline on crop-versus-weed spraying, on two crops, with labels copied from the official dataset files and a held-out set evaluated once (Chapter 6).
2. A deterministic safety gate and a four-rule plant classifier that move the held-out result from 65.5% crop sprayed / 1.8% weed recall to 8.8% / 77.9%, with no training and no model calls (Section 6.4).
3. A bounded escalation agent whose verdicts are combined by measured source accuracy, and a negative but useful result: it is safe and precise but adds no weed recall (Section 6.6).
4. A testing record: an automated suite of 270 tests (235 unit, 35 end-to-end), two system test cycles by a team member on a fresh Windows install, a verification run, a manual test cycle, and 30 findings (F1–F30) traced to fixes or stated limitations (Chapter 7). The final build is released as `v1.0.0`.

1.7 Organisation of the report

Chapter 2 reviews related work and states our base paper. Chapter 3 gives the requirements and the design. Chapter 4 describes the implementation. Chapter 5 describes the data and the evaluation method. Chapter 6 reports the results, Chapter 7 the testing. Chapter 8 discusses the answers to the research questions and the limitations, and concludes. Who built which part is recorded in Appendix B, counted from the repository history.

Chapter 2

Literature review

2.1 Supervised crop–weed segmentation

The classic approach trains a segmentation network on labelled images of the target crop. Milioto, Lottes and Stachniss [5] segment sugar beet and weeds in real time on a field robot; from RGB alone they reach a weed IoU of 20.6%, rising to 59.2% when 14 input channels (RGB, near-infrared and vegetation indices) are used. The sugar beet data of Chebrolu et al. [6] and the pixel-labelled PhenoBench benchmark [7] are the standard datasets of this line of work. On cotton, Rahman et al. trained detectors on CottonWeedDet3 [8], the dataset we use, reaching about 77–80% mAP@0.5 (YOLOv5n 76.6%, RetinaNet 80.0%) after training [9]. These numbers are the ceiling a zero-shot method is compared against, with the caveat that they need labelled training data from the same domain.

2.2 Open-vocabulary detection and promptable segmentation

Grounding DINO [2] fuses an image backbone with a text encoder, so the classes to detect are given at run time as a prompt such as `weed . sugar beet plant .`. The Segment Anything Model [10] and its successor SAM 2 [3] produce a mask for any object given a point or a box, with a predicted quality score, but assign no class. Grounded SAM [1] chains the two: the detector's boxes become the segmenter's prompts, and each mask inherits the box's text label. It reports 48.7 mean AP on the zero-shot SegInW benchmark. Singh et al. [11] found that zero-shot prompting of Grounding DINO struggles with visually similar agricultural classes and that a few-shot adaptation without the text encoder works better.

2.3 Zero-shot crop–weed segmentation

Chong et al. [4] ("Weeds are Weird", WaW) is the work closest to ours. They segment every plant with SAM, describe each plant with BioCLIP [12] features, and treat the crop as the common plant and weeds as outliers, with no labels. They report a mean weed IoU of 12.1% over four datasets (11.1% on PhenoBench) against 62.7% for a supervised ERFNet, and 9.1% when generic CLIP features replace BioCLIP. They explicitly argue that text-prompted pipelines such as Grounded SAM are ill-defined for crop and weed, which is our RQ1. Belissent et al. [13] study transfer and zero-shot weed classification on drone images; their best model, an ImageNet ResNet50 fine-tuned on DeepWeeds, reaches 77.8% accuracy.

2.4 Vision-language models in agriculture

Nasir, Rehman and Hussain [14] test six commercial vision-language models zero-shot on soybean drone images and find weed detection accuracy between 0.15 (LLaMA-4 Scout) and 0.75 (Gemini Flash 2.5): useful as a second opinion, not reliable alone. This motivated our design choice that a language model only advises, never decides, what is sprayed.

2.5 Base paper and research gap

Table 2: Related work and where AgriAgent sits.

Work	Setting	Reported result
Ren et al. 2024, Grounded SAM [1]	Grounding DINO + SAM, zero-shot, general images	48.7 mean AP on SegInW. **The pipeline we implement.**
Chong et al. 2025, WaW [4]	SAM + BioCLIP, zero-shot, crop as the common plant	Mean weed IoU 12.1% vs 62.7% supervised ERFNet. **Closest work and comparison point.**
Milioto et al. 2018 [5]	Supervised CNN, Bonn sugar beet	Weed IoU 20.6% (RGB) / 59.2% (14 channels).
Rahman et al. [8], [9]	Supervised detectors, CottonWeedDet3	About 77–80% mAP@0.5 after training.
Belissent et al. 2024 [13]	Transfer / zero-shot classification, UAV	Best 77.8% accuracy (fine-tuned ResNet50).
Nasir et al. 2026 [14]	Six commercial VLMs, zero-shot	Weed detection accuracy 0.15–0.75.
Singh et al. 2025 [11]	Grounding DINO, few-shot	Zero-shot prompts struggle on similar classes; few-shot adaptation helps.

Base paper. The pipeline AgriAgent implements is Grounded SAM [1], with SAM 2 in place of SAM. Grounded SAM is an arXiv technical report, so we take Chong et al. [4] (IROS 2025) as the peer-reviewed reference point: it addresses the same task, zero-shot, and names our pipeline's weakness. **Gap.** Published zero-shot work reports segmentation quality (IoU). None that we found measures what a spot sprayer needs to know: how much of the crop ends up on the spray map, how many weeds are hit, and whether a transparent safety layer can keep the crop off the map without training. That is what AgriAgent measures.

Chapter 3

Requirements and system design

3.1 Requirements

Table 3: Functional (FR) and non-functional (NFR) requirements.

ID	Requirement
FR1	Accept a JPG or PNG field photo and the crop name; reject empty or corrupt files with a clear message.
FR2	Detect and segment plants zero-shot with Grounding DINO and SAM 2 on the local GPU.
FR3	Spray only zones approved by the deterministic gate; list every withheld plant with its reason.
FR4	Suppress the whole map when the photo is mostly weed (above 85% infestation): spot spraying is the wrong tool there.
FR5	Optionally review withheld plants with the escalation agent; its verdict is advice, the gate still decides.
FR6	Give herbicide advice only from the ICAR bulletin row for the declared crop and weed; otherwise say “no evidence”.
FR7	Export approved zones as GeoJSON [15], one feature per zone, clearly marked as not GPS.
FR8	Summarise the result for a farmer in English, Hindi or Marathi without adding any advice.
NFR1	Honesty: every screen says REAL or MOCK; a real run that cannot load a model fails loudly instead of showing fake output.
NFR2	Safety: any error, missing evidence or disagreement means “do not spray”.
NFR3	Speed: under 1 s per photo on the laptop GPU after the models are loaded.
NFR4	Hardware: fit in 8 GB of GPU memory; reasoning models run in the cloud on free tiers.
NFR5	Testability: deterministic results; all rules unit tested; CI on every push.

3.2 Architecture

AgriAgent follows a “two systems” design. System 1 is fast and local: detection, segmentation, a deterministic gate and rules, all on the laptop. System 2 is slow and deliberate: cloud vision-language models that explain, advise and review, but never decide alone. Figure 1 shows the pipeline.

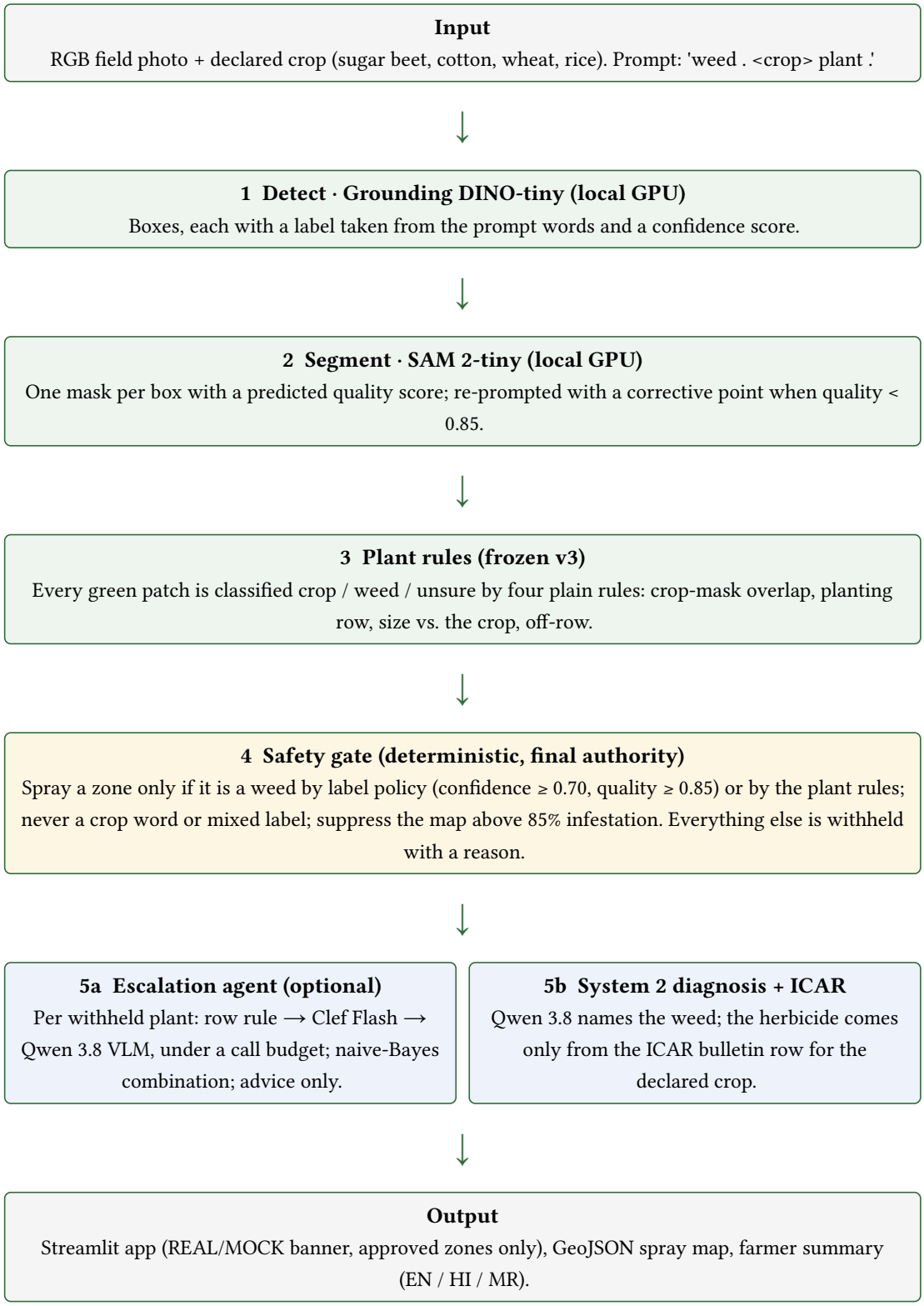


Figure 1: AgriAgent pipeline. Green: local and deterministic or pretrained; yellow: the safety gate; blue: cloud models that advise only.

3.3 The safety gate

The gate is the only component allowed to approve a spray zone (`src/agents/deterministic_router.py`, `src/workflow.py`). Its rules are deliberately plain so that each can be unit tested and explained:

- **Label policy.** A detector mask is a spray candidate only if its label names a weed. Labels that name the crop, or mix crop and weed words (Grounding DINO sometimes returns "weed sugar beet plant"), are withheld.
- **Confidence and quality.** The detector score must be at least 0.70 and SAM 2's predicted mask quality at least 0.85. The app's slider may raise the confidence floor, never lower it.
- **Not one plant.** A mask covering more than half the photo is withheld: it is the whole field, not a weed.
- **Infestation ceiling.** If approved zones cover more than 85% of the photo, the whole map is suppressed.
- **Plant-rule zones.** Patches the plant rules call "weed" are approved; "unsure" patches never are.
- **Fail safe.** Any exception, missing input or missing evidence results in withholding, never spraying.

An early version passed confidence and quality to the gate hard-coded as 1.0, so every "weed" was sprayed; the bug was found in our code review and fixed before any measurement (Section 7.1).

3.4 Plant rules (v3, frozen)

Grounding DINO rarely boxes the small weeds: in our sugar beet photos it boxes mostly the large crop plants (Section 6.2). Its boxes therefore cannot be the only candidates. The plant rules (`src/vision/plant_candidates.py`) take every connected patch of vegetation, found by thresholding the excess-green index ($2G - R - B$) with a per-photo Otsu threshold after closing small gaps between the leaves of one plant, and classify it:

1. **crop** if at least 30% of the patch lies inside a SAM 2 mask the detector labelled as the crop;
2. **crop** if the photo shows clear planting rows and the patch lies on the densest row band;
3. **weed** if the photo has crop masks and the patch is smaller than a quarter of the median crop plant in the same photo (self-calibrating, no threshold tuned on labels);
4. **weed** if the photo shows clear rows and the patch lies off the row band;
5. **unsure** otherwise, which is never sprayed.

The rules were developed on the 30 development photos and then frozen as "v3" before a single run on the 90 held-out photos (Section 5.3). Rule 4 is also the main known weakness (finding F1, Section 6.8).

3.5 The escalation agent

The only agent in AgriAgent (`src/agents/escalation_agent.py`) reviews plants the gate withheld. For each plant it gathers evidence from the cheapest source to the most expensive, the row rule, then Cloudflare's Clef Flash decision model [16], then the Qwen 3.8 27B vision-language model shown a crop of the plant, and stops when the evidence is decisive or its call budget is spent. A source's stated confidence is not its accuracy, so each answer is first capped by the source's measured accuracy a on 20 labelled patches (Section 6.7):

$$P(\text{answer right}) = a \cdot c + (1 - a)(1 - c)$$

where c is the stated confidence. The answers' likelihood ratios are multiplied from even odds, assuming the sources err independently, and the agent says "weed" only when $P(\text{weed}) \geq 0.85$. One source alone can therefore never reach a verdict; two that agree can; sources that disagree cancel out to "unsure". The agent returns a trace of every call, so each verdict can be explained.

3.6 System 2 and ICAR herbicide advice

When the detections are uncertain, or when the user asks, System 2 shows the photo to Qwen 3.8 27B and asks for the crop, the weed and the growth stage. The herbicide then comes only from a table transcribed from the ICAR–DWR herbicide bulletin No. 22 [17] (`src/agronomy/knowledge_store.py`), and only from the row for the declared crop and a matching weed. Doses are clamped to the bulletin's maximum, and banned, withdrawn and refused herbicides (bulletin Table 3.3) are rejected. If no row matches, the screen says "no evidence" instead of guessing. Every row was checked against the PDF (Section 6.9).

3.7 Farmer summary

A short plain-language summary is built only from the pipeline's own facts (zones sprayed, plants left unsure, the prescription). A language model may reword it in Hindi or Marathi, with a fixed glossary for crop and weed words. It is rejected, and the English template shown, if it fails or mentions a dose when no prescription exists. Safety sentences, the wrong-crop warning and "this is a suggestion, read the label", are fixed translated text appended after the model's wording, never left to the model (findings F27 and F29).

Chapter 4

Implementation

4.1 Technology stack and hardware

Table 4: Technology stack.

Layer	Choice
Language	Python 3.11; type hints checked with mypy; style with ruff.
Vision models	`IDEA-Research/grounding-dino-tiny` and `facebook/sam2-hiera-tiny` through Hugging Face Transformers 5.20; PyTorch 2.11 with CUDA 12.8.
Reasoning models	Qwen 3.8 27B (`@cf/qwen/qwen3.8-27b`) and Clef Flash on Cloudflare Workers AI (free tier, 10,000 neurons per day) [16]; Groq [18] as an alternative provider.
Application	Streamlit 1.66 [19]; GeoJSON export [15].
Testing and CI	Python `unittest` ; Streamlit `AppTest` for headless app tests; GitHub Actions on Python 3.10 and 3.11.
Hardware	Laptop with an NVIDIA RTX 5050 (8 GB), 15 GB RAM. Both vision models together peak at 2.4 GB of GPU memory.

4.2 Code structure

Table 5: Main modules.

Module	Responsibility
`src/vision/grounding_dino.py`	Detector wrapper: prompt to boxes; mock mode for tests; fails loudly if real weights cannot load.
`src/vision/sam2_wrapper.py`	Boxes to masks with predicted quality; corrective-point refinement.
`src/vision/model_cache.py`	Loads each real model once per process under a lock (finding F26).
`src/vision/plant_candidates.py`	Plant rules v3 (Section 3.4).
`src/workflow.py`	`VisionWorkflowRunner` : detect, segment, refine, spray map with the gate, triage, System 2.
`src/agents/deterministic_router.py`	Safety gate rules.
`src/agents/escalation_agent.py` , `evidence.py`	Escalation agent and its three evidence sources.
`src/agronomy/`	System 2 reasoner, ICAR knowledge store, prescription and dose ceilings, providers, farmer summary.
`src/evaluation/`	Metrics: crop pixels sprayed, weed recall, savings, IoU/ Dice; benchmark runner.
`app/`	Streamlit dashboard, KPI cards, visualisation, GeoJSON export.
`scripts/`	Baseline runner, manifest builders, provider check, verification scripts.
`tests/`	235 unit tests and 35 end-to-end tests.

4.3 Real and mock modes

Every model wrapper has a mock mode that returns deterministic synthetic output, so that the application and the tests run on any laptop without the 1.5 GB of weights. Early in the project, a failed model load silently switched to mock output, and the application presented a mock diagnosis as "verified". Both were fixed: a real run that cannot load a model now raises an error, and every result screen carries a REAL or MOCK banner. Mock mode never calls a cloud model.

4.4 Application

The Streamlit dashboard (`app/app.py`) takes a photo upload, the crop, the prompt, the confidence slider (0.70 or higher), the nozzle buffer in pixels and two switches: force System 2, and review unsure plants with the escalation agent (real mode, off by default). It shows the REAL/MOCK banner, a wrong-crop warning when no plant matched the declared crop, KPI cards (area sprayed, herbicide saved, zones), the photo beside the spray map with approved zones filled and outlined (tiny zones get a ring so they stay visible), the System 2 report, the farmer summary and a GeoJSON download. An optional view colours every plant by the decision taken (red sprayed, green protected as crop, yellow left alone), so that withheld plants are visible when a result is explained. The last result is cached against the photo and settings, so changing the summary language or downloading does not rerun the models.

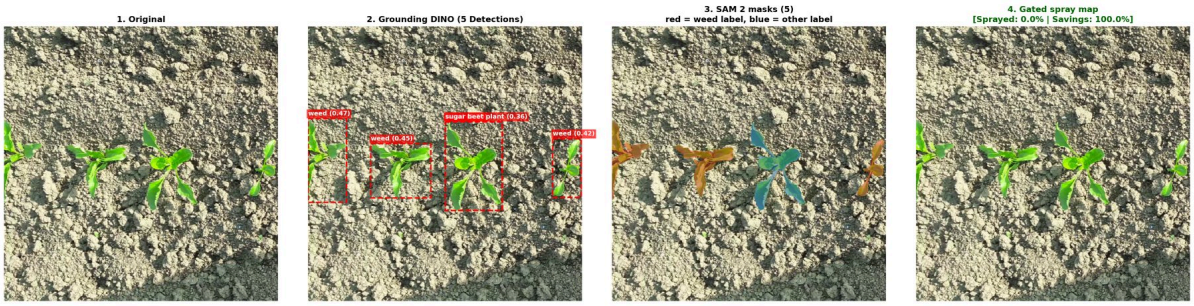


Figure 2: Four-panel view of a real run on a sugar beet photo (17 October, after the gate fix): photo, detector boxes, masks, and the spray map. All detector confidences were 0.36–0.47, below the 0.70 gate, so nothing was sprayed: the crop is safe, but the weeds are missed. This is the Grounded SAM baseline behaviour that the plant rules address.

4.5 GeoJSON export

Each approved zone becomes one MultiPolygon feature traced along the exact pixel boundary of its mask, with holes kept. Fragments under 16 pixels are dropped (jagged mask edges once produced 1,394 features for three weeds). The photo's top-left corner is pinned to a placeholder point at VJTI and each pixel is taken as 5 mm on the ground; image rows grow downward and latitude northward, so rows are subtracted. The file's metadata says, in words, that the coordinates are not GPS. Herbicide fields appear only when System 2 produced a real prescription.

Chapter 5

Data and evaluation method

5.1 Datasets

Table 6: Data sources.

Dataset	What it contains	Used for
sugarbeetsAndweeds [20]	120 photos, 512 × 512, Pascal VOC boxes for sugar beet and weed. Origin not documented by the uploader; the images resemble the Bonn sugar beet data [6].	30 development + 90 held-out
CottonWeedDet3 [8]	Cotton field photos with boxes for three weeds (palmer amaranth, morningglory, waterhemp). Cotton itself is never labelled.	8 development photos
ICAR–DWR bulletin No. 22 [17]	Approved herbicides per crop with dose, water volume, waiting period; banned list.	System 2 knowledge store

5.2 A fabricated evaluation set, found and replaced

The first evaluation set (`eval_manifest_v1`, 20 images) looked real but was a template: every sugar beet image carried the same two crop and six weed boxes, and every cotton image a cotton box, although the official cotton dataset labels no cotton at all. Only one or two of eight boxes per beet image matched the official labels, and none per cotton image (Figure 3). A metric reported on it earlier was meaningless and was withdrawn. It was replaced by `eval_manifest_v2`, built by a script (`scripts/build_eval_manifest_v2.py`) that copies the boxes from the official XML files; rebuilding it from a fresh download gives an identical manifest.



Figure 3: Label audit, beet X-10-0: the old labels (left) put a “crop” box on bare soil and “weed” boxes on beets; official labels (right).

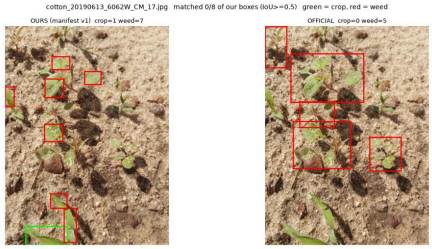


Figure 4: Label audit, cotton CM_17: no old box matches the official weed boxes; the official dataset has no cotton labels.

5.3 Development and held-out sets

Table 7: Evaluation sets. All labels are copied unchanged from the official files.

Set	Photos	Labels	Use
`eval_manifest_v2` (development)	30 beet + 8 cotton	93 beets, 282 weeds; 19 cotton weeds	Every design decision and every variant.
`eval_manifest_v2_holdout`	90 beet	363 beets, 215 weeds	Run **once**, on 18 October, after the plant rules were frozen.

The held-out set is every sugar beet photo not in the development set. It was evaluated once, and the rules were not changed afterwards. A later repeat on 23 October reproduced every number exactly, and a run of the first ten photos twice gave zero differences: the pipeline is deterministic.

5.4 Metrics

Labels are boxes, not pixel masks, so the metrics work at the level of vegetation pixels inside boxes. Vegetation is found with the same excess-green threshold as the plant rules. For a photo:

- **Crop pixels sprayed (%)**: of the vegetation pixels inside crop boxes, the share covered by the spray map. Lower is better; this is the damage measure.
- **Weed recall (%)**: of the vegetation pixels inside weed boxes, the share covered by the spray map. Higher is better.
- **Herbicide saved (%)**: 100 minus the share of the photo that is sprayed, the saving against blanket spraying of the same photo.
- **Plants touched / weeds hit**: the number of labelled boxes with any sprayed vegetation pixel, so that one damaged plant is visible even when the pixel share is small.

Sums are pooled over all photos of a set (not averaged per photo). The metric code (`src/evaluation/crop_protection.py`) has unit tests built from three hand-computed 5 × 5 grids (for example crop 2/6 = 33.3%, weed recall 3/4 = 75%, saving 80%). For cotton, crop pixels sprayed cannot be computed because cotton is not labelled.

5.5 Settings compared

Table 8: Settings compared in Chapter 6. Prompt `weed . <crop> plant .`, detector box threshold 0.35, buffer 10 px.

Setting	Meaning
`every_mask`	Spray every SAM 2 mask, whatever its label: the upper bound on recall and on damage.
`gate_0.35`, `0.50`, `0.70`	Grounded SAM label policy with the confidence floor at 0.35, 0.50 or 0.70 (0.70 is the deployed gate).
`+exclusion`	Also subtract crop-labelled masks from weed masks before spraying (task T1.6).
`plant_rules`	The deployed product: plant rules v3 plus the gate.
agent	Deployed gate plus the escalation agent reviewing up to three withheld plants per photo.

Chapter 6

Results

All runs used the real models on the laptop GPU. Each table names its source file in `docs/results/`.

6.1 Baseline: Grounded SAM label policy (RQ1)

Table 9: Sugar beet development set (30 photos). Source: `docs/results/baseline_v2_20261018.md`.

Setting	Crop sprayed %	Weed recall %	Saved %	Beets touched	Weeds hit
every_mask	93.9	97.6	39.3	87 / 93	265 / 282
gate_0.35	70.8	9.0	93.0	45 / 93	7 / 282
gate_0.50	20.5	6.8	98.0	8 / 93	1 / 282
gate_0.70 (deployed)	0.0	0.0	100.0	0 / 93	0 / 282

SAM 2 outlines plants well: spraying every mask reaches 97.6% weed recall. The problem is the labels. At a confidence floor of 0.35, the label policy sprays 70.8% of crop pixels while hitting only 9% of weed pixels: the detector calls crop plants "weed" more often than it finds the actual weeds. Real Grounding DINO scores on these photos never exceeded 0.68, so the deployed gate at 0.70 sprays nothing at all: perfectly safe and perfectly useless. On cotton, spraying every mask hits 75.8% of weed pixels, but every gated setting hits 0%.

Answer to RQ1: text prompts alone do not separate crop from weed on these photos. Lowering the gate buys recall only at the cost of spraying the crop. This agrees with Chong et al. [4].

6.2 Cheap additions: crop exclusion, lower thresholds, tiling (RQ3)

Table 10: Crop exclusion and detector variants, sugar beet development set. Sources: `exclusion_v2_20261018.md`, `detector_variants_20261018.md`.

Setting	Crop sprayed %	Weed recall %	Beets touched	s / photo
gate_0.35	70.8	9.0	45 / 93	0.58
gate_0.35 + exclusion	50.1	9.0	38 / 93	0.58
gate_0.50	20.5	6.8	8 / 93	0.58
gate_0.50 + exclusion	**5.3**	6.8	2 / 93	0.58
box threshold 0.20, gate_0.50 + exclusion	0.0	0.4	0 / 93	1.03
2 × 2 tiles, gate_0.35	74.6	10.1	49 / 93	3.76

Subtracting crop masks from weed masks (T1.6) cut crop pixels sprayed from 20.5% to 5.3% at a 0.50 floor with no loss of recall. Running the detector on 2 × 2 tiles, or with a lower box threshold, finds more plants but labels them no better, and slows the pipeline up to 13 times. Neither raised weed recall: the bottleneck is classification, not detection. In particular, SAM 2 masks of plant size from detector boxes covered only about 9% of the beet weed pixels; the earlier "97.6%" of `every_mask` comes largely from masks that cover the whole photo.

6.3 Plant rules on the development set

Table 11: Plant rules v3, sugar beet development set. Sources: `plant_rules_v3_20261018.md`, `pipeline_plant_rules_dev_20261020.md` (the product's own code path gives identical numbers).

Setting	Crop sprayed %	Weed recall %	Saved %	Beets touched	Weeds hit
every_plant_mask	84.7	9.1	91.6	53 / 93	8 / 282
gate_0.50 + exclusion	5.3	6.8	99.3	2 / 93	1 / 282
plant_rules (v3)	**4.6**	**71.5**	90.5	21 / 93	219 / 282

6.4 Held-out result (the headline)

Table 12: Held-out sugar beet set (90 photos, evaluated once). Source: `holdout_plant_rules_v3_20261018.md`.

Setting	Crop sprayed %	Weed recall %	Saved %	Beets touched	Weeds hit
every_mask	94.4	93.2	60.2	334 / 363	204 / 215
Grounded SAM, gate_0.35	65.5	1.8	85.0	242 / 363	3 / 215
Grounded SAM, gate_0.50	22.5	0.0	95.5	49 / 363	0 / 215
Grounded SAM, gate_0.70 (deployed)	0.0	0.0	100.0	0 / 363	0 / 215
gate_0.50 + exclusion	5.5	0.0	98.8	24 / 363	0 / 215
plant_rules (v3), the product	**8.8**	**77.9**	95.0	104 / 363	177 / 215

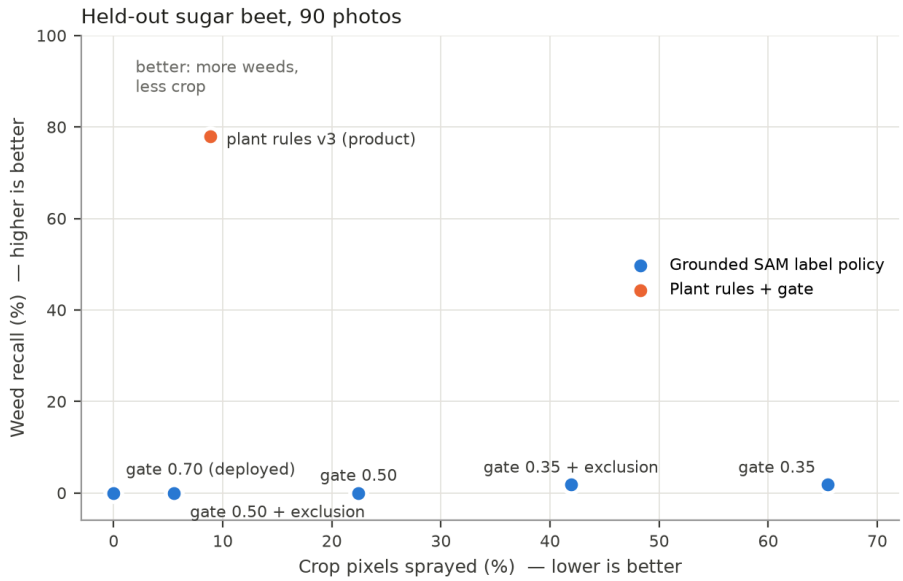


Figure 5: Crop pixels sprayed against weed recall on the held-out set. Every Grounded SAM setting lies on the left edge (almost no weeds hit); the plant rules move the operating point to 77.9% recall at 8.8% crop sprayed.

On photos it had never seen, the product hits 77.9% of weed pixels (177 of 215 weeds) and sprays 8.8% of crop pixels, against 1.8% and 65.5% for the best-recall Grounded SAM setting, while saving 95% of the herbicide of blanket spraying of the same photos. The held-out recall is higher than on the development set (71.5%), so there is no sign of overfitting the rules to the development

photos. The cost is real: 104 of the 363 beets have at least one sprayed pixel, mostly small specks at the edge of a plant, but some whole plants (Section 6.8).

On the eight cotton development photos the rules reach only 10.4% weed recall. Rule 3 relies on weeds being smaller than the crop, true for young sugar beet and false for the cotton close-ups, where a single large weed often fills the photo and is taken for the crop. Cotton therefore remains an open problem for this method.

6.5 Escalation agent (RQ2)

Table 13: Verdicts of the escalation agent on 69 withheld plants, development set, deployed gate. Source: `agent_v2_20261018.md`.

Plant really was	Agent: weed	Agent: crop	Agent: unsure
crop (44)	0	13	31
weed (25)	0	1	24

The agent reviewed up to three withheld plants per photo (69 in total, about 40 minutes, 14 of 69 Groq calls lost to the free tier's per-minute limit and counted as "no answer"). It never approved spraying a crop plant, and 13 of its 14 "crop" verdicts were right. But it called none of the 25 real weeds a weed, so the spray map was identical with and without it.

Answer to RQ2: escalation to vision-language models, combined conservatively, is safe and precise but not effective for the crop-versus-weed decision on this data. The rule-based classifier reaches 77.9% recall with no model calls at all. We keep the agent in the product, off by default, as a reviewer that explains withheld plants.

6.6 Vision-language models as a second opinion

Table 14: Crop-or-weed answers on 20 labelled plant patches. Sources: `t2_2_crop_or_weed_groq_20261017.json`, `qwen_provider_check_20261030.md`.

Source (20 patches: 10 beet crop, 6 beet weed, 4 cotton weed)	Correct	Crop called crop	Weed called weed
Qwen 3.8 27B on Groq, told the crop	16 / 20	9 / 10	7 / 10
Qwen 3.8 27B on Cloudflare, same prompt	12 / 20	5 / 10	7 / 10
Clef Flash (Cloudflare)	14 / 20	8 / 10	6 / 10
Row rule, no model	13 / 20	7 / 10	6 / 10

Twenty patches is a small sample. Two lessons hold anyway. First, the same model name gave 16 of 20 from one provider and 12 of 20 from another (finding F25); a trust weight measured on one provider does not transfer to another, so the agent's weights are tied to the provider they were measured on. Second, in the sugar beet data the weeds are tiny (median 16 px across) and the crops large (median 68 px), so plant size alone separates many cases; this is why the size rule works on beet and not on cotton.

6.7 Main limitation: rule 4 and whole beets (F1)

Rule 4 calls any off-row patch a weed whatever its size. A large beet is saved only when Grounding DINO happens to box it as the crop. When we rotated photo X-10-0 by 90°, the detector returned no crop box and the largest beet (4,410 px) was sprayed completely (Figure 6); photo X2-30-0 shows the same failure on two of three beets. We measured a size limit on rule 4 on the development set only:

Table 15: The size limit trades recall for crop safety. Development set. Source: `scripts/experiment_f1_row_rule.py`, Verification report 23 Oct.

Rule 4 applies to patches up to … × the median on-row plant	**Crop sprayed %**	**Weed recall %**	**Beets mostly sprayed**
0.25	0.25	12.9	0
1.0	0.98	31.3	3
1.5	1.20	39.1	4
no limit (v3, frozen)	4.55	71.5	11

Rule 4 gives most of the recall and most of the damage. We kept v3 frozen, because no unused held-out set is left to test a changed rule fairly, and report the trade-off. A "safe mode" with the size limit is future work.

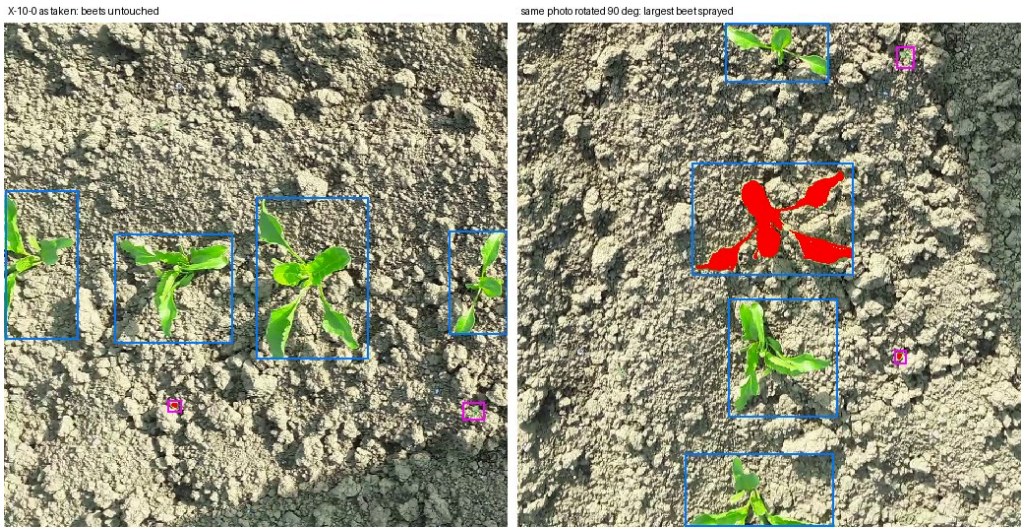


Figure 6: Finding F1. Photo X-10-0 rotated by 90°: the detector returns no crop box, so rule 4 marks the largest beet as an off-row weed and it is sprayed completely.

6.8 ICAR herbicide table check

Every herbicide row and dose ceiling in the code was compared with the rendered pages of the bulletin (`docs/results/icar_table_check_20261020.md`). Eight cotton rows were correct; two had a missing value stored as 0 instead of "not stated". The rice row for pyrazosulfuron-ethyl was dangerous: 20 g a.i./ha with a 75-day waiting period, against the bulletin's 10–15 g and 95 days, an overdose with a shorter waiting period. It and six other values (wheat water volumes and weeds, wheat and rice dose ceilings, an invented beet-root waiting period) were corrected, and all nine banned, withdrawn and refused herbicides matched. Regression tests fail on the old values. In a manual test on cotton photo CM_17, System 2 named the weed *Amaranthus* (correct genus) and matched a real ICAR cotton row: pyrithiobac-sodium 10% EC, 75 g a.i./ha, waiting period 160 days, bulletin page 35.

6.9 Speed

The pipeline takes 0.56–0.64 s per photo on the RTX 5050 after the first photo (detection 0.21 s, segmentation 0.27–0.37 s), well inside NFR3. Loading both models takes 4–14 s; since finding F26 they load once per application process instead of on every run. A real run in the application, including the cloud diagnosis, takes about 15 s, or about 40 s with the escalation agent.

Chapter 7

Testing

7.1 Test levels

Table 16: Testing summary.

Level	What	Result
Unit	235 tests: gate rules, plant rules, metrics (hand-computed grids), ICAR rows vs bulletin, GeoJSON geometry, providers, farmer summary, model cache, verdict view.	all pass
End to end	35 tests through the workflow and the app in mock mode.	all pass
Static	ruff, ruff format, mypy on `src/`.	clean
CI	GitHub Actions on every push, Python 3.10 and 3.11.	green
System, cycle 0 (16 Oct)	Amit Ingle: 16 app test cases on a fresh Windows install.	7 pass, 9 fail → 8 issues, all fixed
Headless app regression	`scripts/verify_cycle0_issues.py`, `scripts/verify_test_cycle1.py`: the app driven by Streamlit AppTest, mock and real mode.	all pass (F11 reworded)
Verification run (23 Oct)	Fresh clone, data rebuild, held-out reproduction, determinism, 12 edge-case images, no-network run.	reproduces exactly; 13 findings
Manual cycle (30 Oct)	MT-01 … MT-10 on six test photos, real mode.	core cases pass; 16 findings
System, cycle 2 (7–8 Nov)	Amit Ingle: the final build on Windows 11, Python 3.13, RTX 3050 (4 GB), real mode: install, automated checks, regression of F24–F29, manual and exploratory cases.	24 of 24 executed cases pass; 1 finding (F30)

7.2 System test cycle 0

On 16 October a team member installed the project from scratch on Windows and ran 16 application test cases. Seven passed and nine failed, giving GitHub issues #5–#12: the app crashed on start because `streamlit run` put the wrong folder first on the import path; GeoJSON files carried herbicide fields with System 2 off; an empty prompt silently became a default prompt that invented weeds; the confidence slider was not applied; mock diagnoses were shown as “verified”; a cotton herbicide was recommended for wheat, rice and sugar beet; and mock mode ignored the uploaded image. Each was reproduced, fixed, covered by a test, re-run with the tester’s exact steps by a headless script, and closed with its cause.

7.3 Findings and fixes

Table 17: Selected findings (full list: `docs/testing/Verification_Report_20261023.md`, `Manual_Test_Cycle_20261030.md`).

ID	Severity	Finding	Status
F1	high	Rule 4 sprays large off-row beets when the detector misses the crop box.	measured; limitation
F2	medium	Unpinned dependencies resolved to untested versions.	fixed: lock file
F3	medium	Groq free tier rate limit (429) on back-to-back runs.	fixed: Cloudflare, agent off by default
F4	medium	Farmer summary's "unsure" count used gate-withheld masks.	fixed
F5	medium	Tiny approved patches invisible on screen.	fixed: outline and ring
F14–F16	medium	GeoJSON upside down; buffer labelled cm but in pixels; 1.1 m per pixel.	fixed
F17, F19	medium	Silent fall-back to English.	fixed: note on screen
F18	high	Wrong crop setting: no crop masks, so rule 4 sprays crop plants.	fixed: warning
F20	medium	Every click reran the models.	fixed: result cache
F22	medium	Full-size photos sent to the cloud model (12,349	fixed: resized
F24	medium	[illegible]	safe: the crop label is [illegible] why labels need a gate

7.4 Manual test cycle

The manual cycle used six photos with official labels (four sugar beet, two cotton). Representative outcomes: on beet X3-130-0 the product sprays 0.05% of the photo with no beet filled (MT-02); with the crop wrongly set to cotton the wrong-crop warning appears and one beet is sprayed, the known F1 failure (MT-03); the escalation agent's reviews all completed on Cloudflare without rate-limit errors (MT-08); on cotton CM_17 the detector boxed the three large palmer amaranth weeds as cotton, so the gate protected them and only 0.06% was sprayed, while System 2 produced the first real ICAR prescription on a test photo (MT-09). Results are recorded in `docs/testing/Manual_Test_Cycle_20261030.md`.

7.5 System test cycle 2 and the release

On 7–8 November Amit Ingle installed the final build from scratch on his own laptop (Windows 11, Python 3.13.9, NVIDIA RTX 3050 with 4 GB, real mode with Cloudflare keys, commit `b9f6364`) and ran the 12-page Test Cycle 2 guide (`docs/testing/Test_Cycle_2_Brief.pdf`). All 24 executed cases passed: the install and automated checks (A1–A6: 270 tests, 35 end-to-end tests, the headless app checks in mock and real mode, both cloud providers), the regression of the 7 November fixes (R1–R6, F24–F29), four manual cases never run before (MT-04 to MT-07, including the GeoJSON on geojson.io and the known F1 failure) and eight exploratory cases (X1–X8: an unrelated flower photo, a photo with no plants, very large and very small images, transparent PNGs, two browser tabs at once, controls changed during a run, no network, and a screenshot of a text document). The first real run took 3.6 s on the 4 GB GPU.

Before the cycle, preparing it exposed a release blocker: the lock file pinned CUDA libraries, `triton` and `uvloop` that exist only for Linux, so the documented install failed on Windows. They are now marked Linux-only, and the lock file was checked to resolve for Windows on Python 3.11–3.13 (Python 3.10 cannot install it, so the project now requires 3.11). The exploratory case X8 gave finding F30: Grounding DINO labelled a page of text "sugar beet plant" at 0.71 confidence. The gate sprayed nothing because the label names the crop, a small illustration of RQ1: detector labels are not evidence. Amit then prepared the release (version bump, release notes) and reported a bug in the packaging script (a branch name with `/` broke the file name), which was fixed in review. The final build is tagged `v1.0.0`; its source package is built by `scripts/package_release.py` from the tag and checked for secrets.

Chapter 8

Discussion and conclusion

8.1 What we learned, including what went wrong

1. **AI-written status claims.** AI-generated documents early in the project described intentions as facts: "100% complete", 227 tests (there were 175), a "0.99 IoU" that was SAM 2's own confidence score. Since 11 October every claim carries a status label (REAL, PARTIAL, MOCK, PLANNED) and a command output.
2. **Fabricated labels.** The first evaluation set was a template (Section 5.2). Ground truth is now copied from the official files by a script and audited.
3. **Silent fake output.** Failed model loads became mock output; every SAM 2 mask got an invented quality of 0.88; the reasoning model never received the photo. Real mode now fails loudly.
4. **A gate that ignored its inputs.** Confidence and quality were passed as 1.0. Each gate rule now has a test with real values.
5. **Measure before claiming.** The original proposal promised "over 70% chemical saving" and "under 40 ms". Neither was measured then; the measured numbers are in Chapter 6.

8.2 Answers to the research questions

- **RQ1.** Grounded SAM's text labels do not separate crop from weed: on held-out photos 65.5% of crop pixels are sprayed for 1.8% weed recall, and the deployed 0.70 gate sprays nothing. SAM 2's masks are good; the labels are the problem.
- **RQ2.** The escalation agent is safe (no crop approved) and precise (13 of 14 crop verdicts right) but adds no weed recall (0 of 25 weeds).
- **RQ3.** Crop exclusion cuts damage without recall gain; lower thresholds and tiling do not help. Plain rules on crop overlap, rows and relative size give the best trade-off measured: 8.8% crop sprayed and 77.9% weed recall held out.

8.3 Relation to the base paper and the closest work

We implemented the Grounded SAM pipeline [1] and confirm on field photos the weakness Chong et al. [4] argue for: text prompts are ill-defined for crop and weed. We cannot claim to beat WaW's 12.1% weed IoU, because we measure a different quantity (sprayed vegetation inside labelled boxes, not pixel IoU against pixel masks) on different data. A like-for-like comparison would run our pipeline on the PhenoBench validation set (772 pixel-labelled images) and report weed IoU; it is the first item of future work. What we add is the sprayer's view: how much crop lands on the map, how many weeds are hit, and how far a transparent, deterministic safety layer moves that trade-off without training.

8.4 Limitations

- Small data: 120 sugar beet photos from one dataset of undocumented origin, 8 cotton photos.
- Box labels only, so pixel metrics are approximate (vegetation inside boxes).
- The plant rules exploit "weeds are smaller than the crop", true for young sugar beet; they reach only 10.4% weed recall on cotton.

- Finding F1: whole off-row beets can be sprayed when the detector misses the crop box (104 of 363 held-out beets touched, mostly specks).
- The held-out set has been used, so any changed rule can only be tested on new data.
- Coordinates are not GPS; no sprayer, latency or field trial was tested.
- Cloud reasoning depends on free tiers and on the provider (F25).

8.5 Future work

1. Weed IoU on PhenoBench for a direct comparison with WaW [4].
2. Species-aware features (BioCLIP [12]) instead of text labels, as WaW suggests, and a "safe mode" size limit on rule 4, both tested on new held-out data.
3. Few-shot adaptation of the detector [11] with a handful of labelled field photos.
4. A farmer-facing conversational agent over the same facts, a mobile web interface, and the other applications of the proposal (disease, yield, canopy, nutrients).
5. Real GPS from drone metadata and a field trial with an agronomist.

8.6 Conclusion

AgriAgent shows, with official labels and a held-out set, that a zero-shot Grounded SAM pipeline cannot on its own tell a weed from a crop plant well enough to spray: it either sprays the crop or nothing. A small set of transparent rules and a deterministic safety gate, with no training and no model calls, move the held-out result to 77.9% weed recall at 8.8% crop pixels sprayed, in 0.6 s per photo on a laptop. A vision-language escalation agent, used conservatively, stays safe but does not add recall. The system is honest about what it is: a research prototype that measures how far zero-shot vision gets on this problem, not a product for real spraying.

References

[1] T. Ren *et al.*, "Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks," *arXiv preprint arXiv:2401.14159*, 2024, [Online]. Available: https://arxiv.org/abs/2401.14159

[2] S. Liu *et al.*, "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection," in *European Conference on Computer Vision (ECCV)*, 2024. [Online]. Available: https://arxiv.org/abs/2303.05499

[3] N. Ravi *et al.*, "SAM 2: Segment Anything in Images and Videos," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2408.00714

[4] Y. L. Chong, L. Nunes, F. Magistri, X. Zhong, J. Behley, and C. Stachniss, "Zero-Shot Semantic Segmentation for Robots in Agriculture," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. [Online]. Available: https://github.com/PRBonn/WeedsAreWeird

[5] A. Milioto, P. Lottes, and C. Stachniss, "Real-time Semantic Segmentation of Crop and Weed for Precision Agriculture Robots Leveraging Background Knowledge in CNNs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018. [Online]. Available: https://arxiv.org/abs/1709.06764

[6] N. Chebrolu, P. Lottes, A. Schaefer, W. Winterhalter, W. Burgard, and C. Stachniss, "Agricultural robot dataset for plant classification, localization and mapping on sugar beet fields," *International Journal of Robotics Research*, vol. 36, no. 10, 2017.

[7] J. Weyler *et al.*, "PhenoBench: A Large Dataset and Benchmarks for Semantic Image Interpretation in the Agricultural Domain," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[8] A. Rahman, Y. Lu, and H. Wang, "CottonWeedDet3: A 3-class weed detection dataset for cotton cropping systems." [Online]. Available: https://www.kaggle.com/datasets/yuzhenlu/cottonweeddet3

[9] A. Rahman, "Performance evaluation of deep learning object detectors for weed detection and real time deployment in cotton fields," Master's thesis, 2024.

[10] A. Kirillov *et al.*, "Segment Anything," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. [Online]. Available: https://arxiv.org/abs/2304.02643

[11] R. Singh, R. Bidese Puhl, K. Dhakal, and S. Sornapudi, "Few-Shot Adaptation of Grounding DINO for Agricultural Domain," *arXiv preprint arXiv:2504.07252*, 2025, [Online]. Available: https://arxiv.org/abs/2504.07252

[12] S. Stevens *et al.*, "BioCLIP: A Vision Foundation Model for the Tree of Life," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[13] N. Belissent, J. M. Peña, G. A. Mesías-Ruiz, J. Shawe-Taylor, and M. Pérez-Ortiz, "Transfer and zero-shot learning for scalable weed detection and classification in UAV images," *Knowledge-Based Systems*, vol. 292, p. 111586, 2024, doi: 10.1016/j.knosys.2024.111586.

[14] M. F. Nasir, M. U. Rehman, and I. Hussain, “Vision-language models for zero-shot weed detection and visual reasoning in UAV-based precision agriculture,” *Frontiers in Plant Science*, vol. 16, p. 1735096, 2026, doi: 10.3389/fpls.2025.1735096.

[15] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, and T. Schaub, “The GeoJSON Format,” *IETF RFC 7946*, 2016, doi: 10.17487/RFC7946.

[16] Cloudflare, “Workers AI documentation: models @cf/qwen/qwen3.8-27b and @cf/cloudflare/clef-flash.” 2026.

[17] ICAR–Directorate of Weed Research, “Herbicide use bulletin No. 22.”

[18] Groq, “GroqCloud documentation: model qwen/qwen3.8-27b.” 2026.

[19] Snowflake Inc., “Streamlit documentation.” 2026.

[20] wangyongkun, “sugarbeetsAndweeds.” [Online]. Available: https://www.kaggle.com/datasets/wangyongkun/sugarbeetsandweeds

Appendix A: How to run

```
git clone https://github.com/Chaitanya-666/agriAgent && cd agriAgent
python3.11 -m venv ~/.venvs/agriagent && source ~/.venvs/agriagent/bin/
activate.fish   # Python 3.11–3.13
pip install -r requirements-lock.txt && pip install -e . --no-deps   # Linux
and Windows
python scripts/download_models.py            # Grounding DINO-tiny + SAM 2-
tiny
cp .env.example .env                          # add CLOUDFLARE_ACCOUNT_ID and
CLOUDFLARE_API_TOKEN
streamlit run app/app.py                      # dashboard
PYTHONPATH=. python -m unittest discover -s tests          # unit tests
python scripts/run_baseline.py --manifest data/manifests/
eval_manifest_v2_holdout.json  # held-out run
```

Appendix B: Team contributions

The numbers below are counted from the repository, not estimated: commits without merge commits (`git log`), the current Python code by its last author (`git blame` over every tracked `.py` file in `src/`, `app/`, `tests/` and `scripts/`), and pull requests and issues on GitHub. The same record is kept in the repository as `CONTRIBUTIONS.md`, regenerated by `scripts/contributions.py`. Counted on 8 November 2026 at commit `8c4afe7` (release `v1.0.0`).

Table 18: Contributions counted from the repository.

Member	Commits	Current code (lines)	Pull requests	Issues
Chaitanya Shinde (lead)	100 of 106	16,220 (97.4%)	25	1
Amit Ingle	4	225 (1.4%)	2	8
Sumant Vetal	1	210 (1.3%)	1	0
Sahil Chavan	1	0	1	0

Table 19: Components and their owners (owner = who built the current version).

Component	Owner	Note
Research questions, scope and methodology	Chaitanya	
Vision pipeline: Grounding DINO, SAM 2, refinement, model loading	Chaitanya	
Deterministic safety gate and spray map	Chaitanya	
Plant rules v3	Chaitanya	
Escalation agent and its evidence sources	Chaitanya	
System 2: diagnosis, ICAR knowledge store, prescription, providers	Chaitanya	first version by Sumant (5 Oct); rewritten, checked against the bulletin and integrated by Chaitanya
Farmer summary in English, Hindi and Marathi	Chaitanya	
Streamlit application	Chaitanya	first dashboard by Amit (4 Oct)
GeoJSON export	Chaitanya	
Evaluation sets from official labels, label audit, metrics	Chaitanya	dataset downloader by Sahil (4 Oct)
All experiments and results (Chapter 6)	Chaitanya	
Unit and end-to-end tests, CI, verification run, manual test cycle	Chaitanya	
System test cycle 0 (16 cases, fresh Windows install, 8 issues)	Amit	
System test cycle 2 on the final build (7–8 November: 24 of 24 cases pass on Windows)	Amit	
Release v1.0.0: version bump, release notes, draft release; packaging bug report	Amit	notes and packaging fix finished in review by Chaitanya
Fixes for all findings F1–F30	Chaitanya	
This report, the slides, the guides and all project documents	Chaitanya	

In short: Chaitanya Shinde designed and built the system, ran every measurement, fixed every finding and wrote this report and the slides. Amit Ingle built the first dashboard, ran the system test cycles and prepared the `v1.0.0` release. Sumant Vetal wrote the first version of the System 2 code. Sahil Chavan wrote the dataset downloader.